Medición experimental de tiempos de ejecución en programas.

USC, Grado de ingeniería informática. Programación II,

Marcelo Fort Muñoz.

Curso 2021-22

1. Introducción al problema:

Ante la problemática descrita de calcular los tiempos de ejecución de dos operaciones sobre unas matrices diseñadas e implementadas en un TAD de creación propia, se redacta este informe.

Se nos ha planteado la medición de tiempos de ciertas operaciones llevadas con matrices cuadradas (n x n) de distinto tamaños. Para llevar a cabo estas pruebas, se ha diseñado un TAD de matrices con el que hacer las operaciones.

Los tiempos se medirán a través de la función `clock(),` que se encuentra en la librería `time.`h. Esta función obtiene el número de ticks que han ocurrido desde la creación del proceso que lo alberga, y, de esta forma al restar el `clock()` siguiente a uno previo, se puede saber cuántos ticks ha dado el reloj en ese periodo de tiempo específico y, si lo dividimos entre el número de ticks por segundo, que se almacena en la constante `CLOCKS_PER_SEC` (también albergada en `time.`h), obtenemos el tiempo transcurrido entre operación y operación.

Tras obtener estos resultados, se representarán gráficamente, gracias a unos scripts de Matlab, los resultados obtenidos. Estos scripts mostrarán el valor de los distintos n utilizados en el eje x y los tiempos de ejecución en el eje y.

Además, para cada una de las funciones se obtendrá una curva cuadrática o cubica (según el caso) que aproxime a dichas curvas los tiempos obtenidos en cada operación.

2. Pruebas realizadas:

Para el caso de la suma de matrices cuadradas hemos hecho pruebas con matrices que van desde n = 200 hasta n = 12000 con incrementos de 200 unidades. No hemos podido avanzar más allá, puesto que nos encontrábamos con un error de memoria a partir de n = 13000.

Para el caso de la multiplicación de matrices cuadradas hemos hecho pruebas con matrices que van desde n = 200 hasta n = 5000 con incrementos de 200 unidades. Se ha decidido detener las pruebas en n = 5000 por el tiempo que tardaba este en computarse, siendo mayor a 30 minutos.

3. Resultados obtenidos:

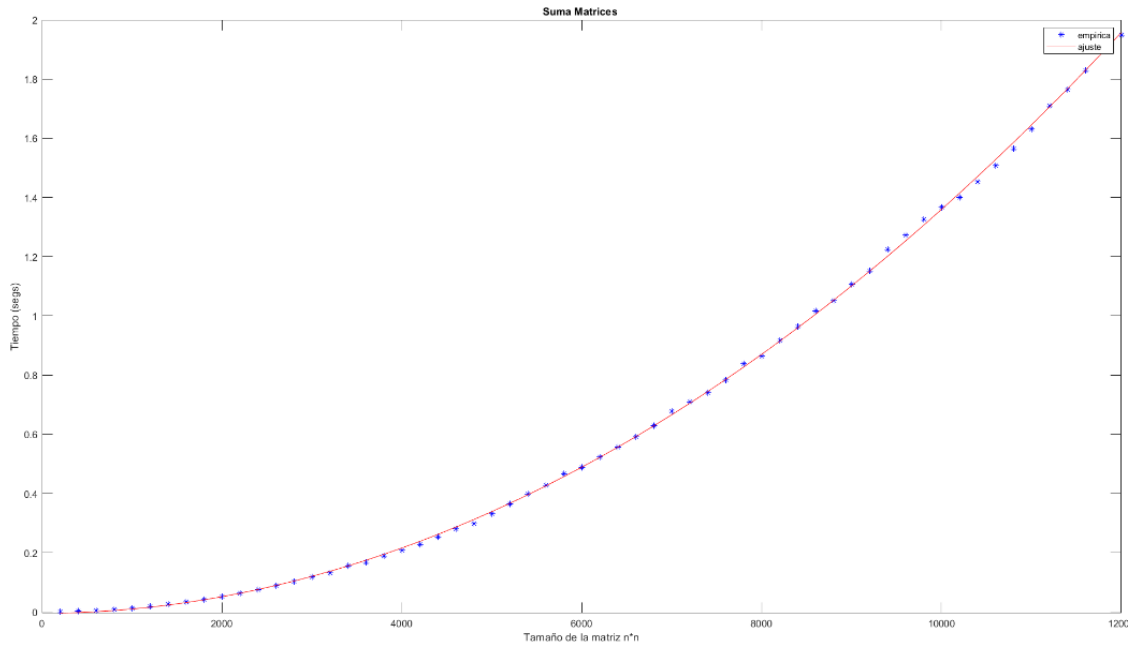
Tras ejecutar el código y los scripts de Matlab que nos muestran las representaciones gráficas de los tiempos de las operaciones, hemos obtenido una curva para cada una de las funciones estudiadas.

Suma de matrices

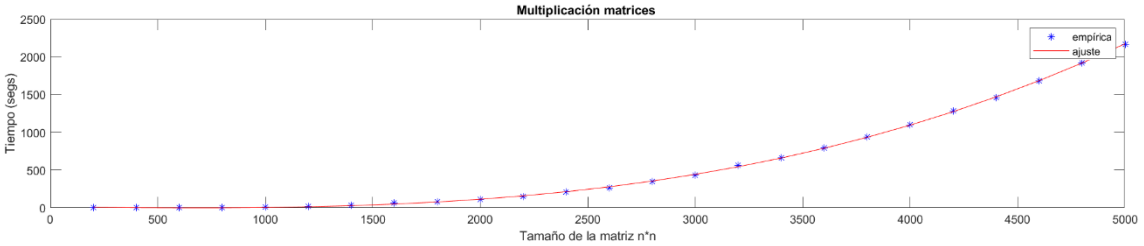
Para la suma se han calculado tamaños hasta 12000, puesto que en torno a este valor el programa se queda sin memoria y ya no es capaz de generar nuevas matrices. Los tiempos medidos tienen un tiempo razonable, siendo el más alto medido 1.949 segundos en n = 12000.

En la gráfica 1, que representa el tiempo requerido para la realización de la suma de matrices cuadradas, se puede observar una curva cuadrática de fondo en color rojo con puntos azules superpuestos.

Cada punto azul representa una operación realizada en el tiempo y con un tamaño específico para las matrices, incrementando el tamaño de estas en 200 en cada iteración. Se puede observar que los datos tomados de forma experimental que se representan como puntos azules se alinean muy bien con la curva representada en el fondo. Esto implica que la complejidad de esta operación matricial es de $O(n^2)$.



Gráfica 1. . Tiempo de la suma de 2 matrices cuadradas



Gráfica 2. Tiempo de la multiplicación de 2 matrices cuadradas.

Multiplicación de matrices

Al principio, el tiempo es prácticamente t(n) = 0, pero según va incrementándose el tamaño de las matrices, se va incrementando el tiempo, hasta volverse insostenible en n = 5000, que llega a tardar 2169.307 segundos (es decir, más de 36 minutos).

La operación deja de ser razonable en n = 600, cuando sube de los 2 segundos y adquiere un tiempo de ejecución que no es realista en la mayoría de situaciones en un entorno real.

Para la gráfica 2, observamos la gráfica relativa a la operación de multiplicar matrices, que está compuesta, por los valores obtenidos experimentalmente y por una curva que aproxima a una curva cúbica los puntos obtenidos.

Como en la gráfica 1, cada operación realizada se muestra como un punto azul y, estos se alinean casi perfectamente con la curva cúbica, por lo que se puede analizar que la operación multiplicación de matrices tiene una complejidad de $O(n^3)$.

Conclusión

Y he aquí los hechos por los que se concluye que, tras analizar los datos de las dos gráficas, para toda la muestra desde n=200 hasta n = 5000 es más eficiente la operación de suma de matrices, dados los

t(n) que arroja al realizarse. Esto ocurre porque esta operación tiene menor complejidad, por lo que gasta menos ciclos de reloj, ergo, menos tiempo.

4. Respuesta a las preguntas del enunciado

Aunque entiendo que ya estaban respondidas en el documento, respondo aquí específicamente a las preguntas que se hacían en el enunciado.

1. ¿Qué tamaños de matrices son gestionables en tiempo "razonable" por cada algoritmo?

Para el algoritmo de suma todos los tiempos son gestionables en un tiempo adecuado, de hecho, el programa se queda sin memoria en n=12000 antes de que el tiempo de cálculo se vuelva excesivo.

Sin embargo, en el caso del producto de matrices, el tiempo se vuelve poco gestionable en n = 5000, cuando sube de los 2 segundos, ya que este tiempo no es admisible en la mayoría de casos en una aplicación con más operaciones.

2. ¿Es más eficiente en general alguna de las dos operaciones? En caso afirmativo, indica si es así para todos los valores de n o únicamente para algunos. En este último caso, indica el rango de valores de n para los que es más ágil cada operación.

La suma, al tener una complejidad menor ($O(n^2)$) es mucho más eficiente que la multiplicación ($O(n^3)$). Esto ocurre en todo n medido en el rango en que hemos podido probar ambas funciones (entre n = 200 y n = 5000).

Apéndice: Datos Experimentales.

Aquí se muestran los datos obtenidos experimentalmente.

I. Suma de matrices:

n	t(n), en segundos
200	0.000000
400	0.002000
600	0.004000
800	0.008000
1000	0.013000
1200	0.018000
1400	0.026000
1600	0.034000
1800	0.042000
2000	0.052000
2200	0.063000
2400	0.075000
2600	0.088000
2800	0.102000
3000	0.118000
3200	0.132000
3400	0.156000
3600	0.167000
3800	0.188000
4000	0.208000

n	t(n), en segundos
4200	0.227000
4400	0.253000
4600	0.280000
4800	0.298000
5000	0.331000
5200	0.365000
5400	0.399000
5600	0.427000
5800	0.467000
6000	0.489000
6200	0.523000
6400	0.557000
6600	0.592000
6800	0.630000
7000	0.678000
7200	0.709000
7400	0.741000
7600	0.783000
7800	0.839000
8000	0.863000

n	t(n), en segundos
8200	0.917000
8400	0.963000
8600	1.017000
8800	1.051000
9000	1.107000
9200	1.152000
9400	1.224000
9600	1.272000
9800	1.325000
10000	1.366000
10200	1.400000
10400	1.453000
10600	1.507000
10800	1.565000
11000	1.631000
11200	1.709000
11400	1.765000
11600	1.828000
11800	1.910000
12000	1.949000

II. Multiplicación de matrices:

n	t(n), en segundos
200	0.085000
400	0.619000
600	2.166000
800	5.074000
1000	10.479000
1200	18.367000
1400	31.923000
1600	61.596000
1800	77.985000
2000	108.557000

n	t(n), en segundos
2200	150.361000
2400	208.229000
2600	266.506000
2800	348.043000
3000	432.914000
3200	561.731000
3400	662.793000
3600	795.326000
3800	936.886000
4000	1098.202000

n	t(n), en segundos
4200	1285.905000
4400	1464.658000
4600	1681.969000
4800	1922.563000
5000	2169.307000